



Database Forensics

How Data Solves Crimes in Large Organizations

**A 5 - slide guide to investigating data
at scale with SQL and audit logs**

EVODIA UNJUEH AKU
unjuehevo@gmail.com

Day 9 of 30 | June 2026





1'S COMPLEMENT REPRESENTATION

Why do we need complements?

Sign-magnitude representation has two major problems:

Two representations of zero (+0 and -0)

Arithmetic operations are complicated

What is 1'S COMPLEMENT Representation?

Positive Numbers

Positive numbers are represented normally.

Definition of 1's Complement

1's complement is obtained by changing every bit of a binary number to its opposite.

That is by flipping all bits

Every 0 becomes 1

Every 1 becomes 0

Example 4-bit representation +5 = 0101

1's Complement = 1010

RULE: Change all zeros to ones and all ones to zero

In 1's complement, flipping is used only to **create or interpret negative numbers**. The MSB alone does not trigger flipping; it only indicates whether the number is positive or negative.

CONCEPT

Negative Numbers

Negative numbers are obtained by taking the **1's complement of the positive number**.

Rule:

To get the negative of a number → flip all bits

The computer is NOT thinking:

“this is a sign bit”

It is thinking:

“this is just a binary number, but negative numbers are represented by flipping bits”





1'S COMPLEMENT REPRESENTATION

Decimal Value	MSB (Most Significant Bit)	1'S COMPLEMENT	Binary Representation
+0	0	0000 (4-bit)	0000
-0000 = -0	1	1111	in binary we only have one 0
-6	1	1001	0110
+6	0	0110	0110
+3	0	1100	0011
+15	-	in 4-bit it will cause an overflow error	1111 binary is = 15

General rule

Positive answer (MSB = 0)

Just convert normally.

Negative answer (MSB = 1)

Take the 1's complement again.

Convert to decimal.

Attach the negative sign

1111

- ☐ unsigned integer representation is = 15
- ☐ sign- magnitude representation = -7
- ☐ 1's complement representation = -0

RANGE

4-bit -7 to +7

8-bit -127 to + 127





OPERATIONS, End-around carry AND OVERFLOW

4-BIT RANGE -7 TO +7
RAGNGE FOR 8-BITS = -127 TO +127

4-bit
5 + 2
2 - 5

+5 = 0101
+2= 0010
+5 + (+2)
0101
0010
0111

8-bit
9 - 4
4 - 9

MSB = 0 = POSITIVE
+7

4-bit
5 + 3

-5 FLIP THE BITS OF +5 = 1010
+2 + (-5)
0010
1010
1100

8-bit
-3 - 5

MSB = 1 = NEGATIVE and since it is negatives we note the sign as negative and flip the bits to get the actual value
0011
-3

+ 9 = 1001 = 00001001
+4 = 0100 = 00000100
-4 flip the bits of +4 = 11111011

+9 +(-4)
00001001
11111011
00000100
we have an extra 1 so we add to the LSB this is called
END-AROUND CARRY

00000100
1
00000101
MSB = 0 = POSITIVE
+5

+4 + (-9)
00000100
11110110
11111010
MSB = 1 = NEGATIVE
SO FLIP THE BITS
00000101
-5

+5 + (+3)
0101
0011
1000
MSB = 1 = -VE
0111
-7

The values are in the range but +8 the answer exceeds the range and this cause data corruption. FATAL ERROR

(-3) + (-5)
+3 = 00000011 flip to get -3 = 11111100
+5 = 00000101 flip to get -5 = 11111010
11111100
11111010
11110110
END-AROUND CARRY 1 so we add to the LSB
11110110
1
11110111 MSB = -VE FLIP THE BIT
00001000 = -8





TERMS CHEAT SHEET

- ☐ DOUBLE ZERO PROBLEM
 - ☐ ADDER CIRCUIT
 - ☐ OVERFLOW
 - ☐ SOLUTIONS
-
- ☐ Status Register or Flags register (Overflow flag or v-flag)
-
- ☐ Crash/Throw an Exception
 - ☐ Wrap around silently
-
- ☐ Upgrade the datatype (Perform sign extension)
-
-
- ☐ Saturation Arithmetics e.g audio processing
-
- ☐ Upgrading to 2'S COMPLEMENT

EXPLORE THE FULL REPO ON GITHUB.

